

Київський національний університет будівництва і архітектури

Кафедра КБЗІ

МАГІСТРАТУРА

Лабораторна робота №1

Тема: Архітектура ROS2. Реалізація Publisher / Subscriber та запуск прикладу
turtlesim

Виконав студент КСМм-26:
Павлюк О.А.

Перевірив
Маринський К.В.

Київ – 2026

Мета:

Сформуванати розуміння архітектури ROS2, принципів роботи нод, топіків, механізму Publisher/Subscriber та навчитися створювати власний ROS2-пакет і реалізовувати обмін повідомленнями між нодами.

Завдання:

1. Перевірка встановлення ROS2 та запуск прикладу

1.1. Перевірити коректність встановлення ROS2.

1.2. Запустити приклад turtlesim.

1.3. Запустити teleop для керування черепахою.

1.4. За допомогою команд `ros2 node list` та `ros2 topic list` визначити активні ноди та топіки.

1.5. За допомогою `ros2 topic echo` переглянути повідомлення одного з топіків.

2. Створення власного пакету

2.1. Створити ROS2-пакет з типом збірки `ament_python`.

2.2. Зібрати workspace та активувати середовище.

2.3. Перевірити коректність встановлення створеного пакету.

3. Реалізація Publisher

3.1. Створити ноду, що публікує текстові повідомлення у Topic з використанням `std_msgs/String`.

3.2. Повідомлення повинно публікуватися з фіксованою періодичністю (1 раз на секунду).

3.3. Додати логування (INFO-рівень) при кожній публікації.

4. Реалізація Subscriber

4.1. Створити окрему ноду-підписника.

4.2. Підписатися на створений Topic.

4.3. Реалізувати вивід отриманого повідомлення у консоль через логер ROS2.

5. Перевірка роботи системи

5.1. Запустити Publisher та Subscriber в окремих терміналах.

5.2. Переконаватися у коректному обміні повідомленнями.

5.3. Перевірити відображення нод і топіків через стандартні команди ROS2.

Звіт

1) За офіційною інструкцією було встановлене середовище ROS 2 Lyrica на Linux Ubuntu на віртуальній машині (<https://docs.ros.org/en/lyrical/Get-Started/Installation/Ubuntu-Install-Debs.html>)

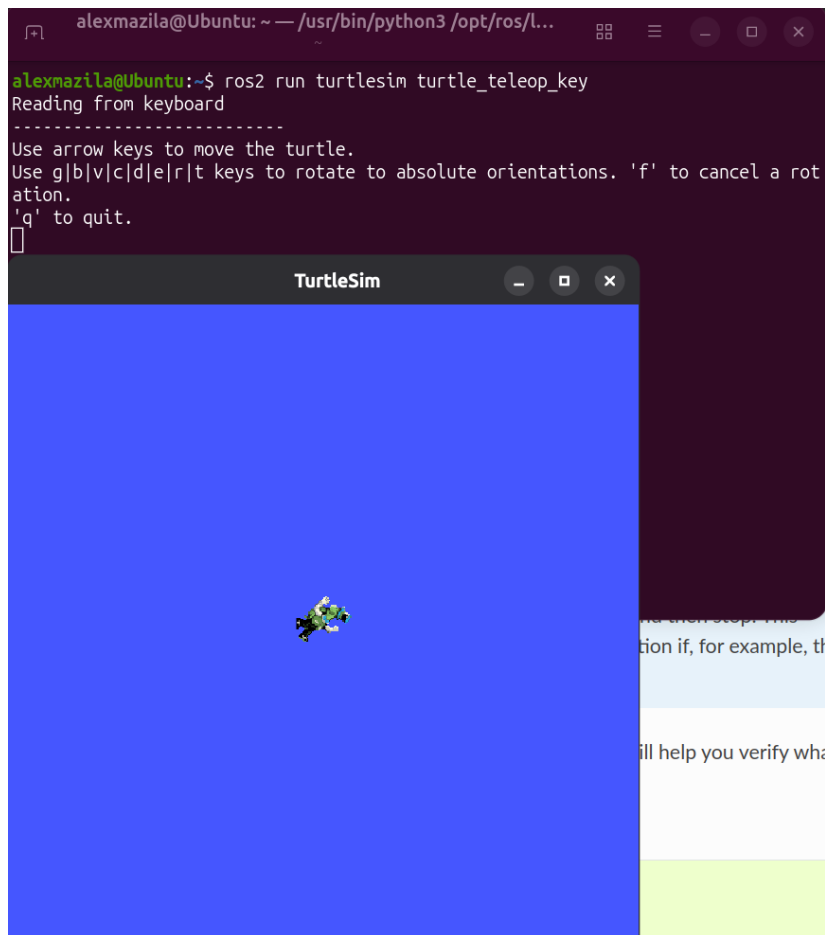
Для перевірки правильності встановлення було виконано декілька команд:

```
alexmazila@Ubuntu: ~  
alexmazila@Ubuntu:~$ printenv | grep -i ROS  
ROS_VERSION=2  
ROS_PYTHON_VERSION=3  
AMENT_PREFIX_PATH=/opt/ros/lyrical  
CMAKE_PREFIX_PATH=/opt/ros/lyrical/opt/gz_math_vendor:/opt/ros/lyrical/opt/gz_utils_vendor:/opt/ros/lyrical/opt/gz_cmake_vendor  
ROS_AUTOMATIC_DISCOVERY_RANGE=SUBNET  
PYTHONPATH=/opt/ros/lyrical/opt/gz_math_vendor/lib/python:/opt/ros/lyrical/lib/python3.14/site-packages  
LD_LIBRARY_PATH=/opt/ros/lyrical/opt/gz_math_vendor/lib:/opt/ros/lyrical/opt/gz_utils_vendor/lib:/opt/ros/lyrical/opt/rviz_ogre_vendor/lib:/opt/ros/lyrical/lib/x86_64-linux-gnu:/opt/ros/lyrical/opt/gz_cmake_vendor/lib:/opt/ros/lyrical/lib  
PATH=/opt/ros/lyrical/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin:/snap/bin  
ROS_DISTRO=lyrical
```

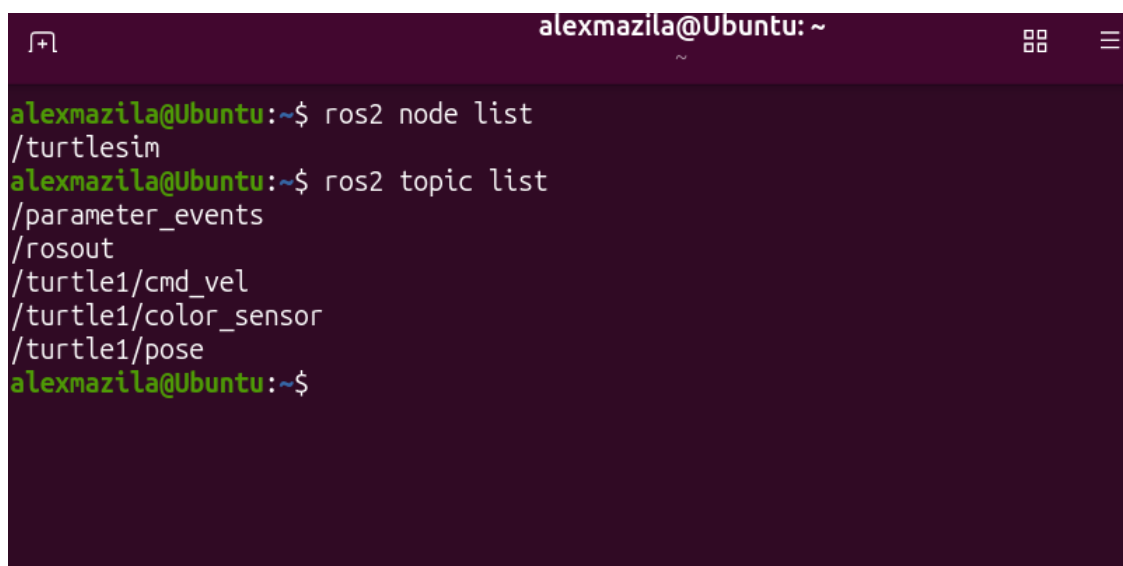
Для демонстрації можливостей середовища завантажуюмо пакети turtlesim:

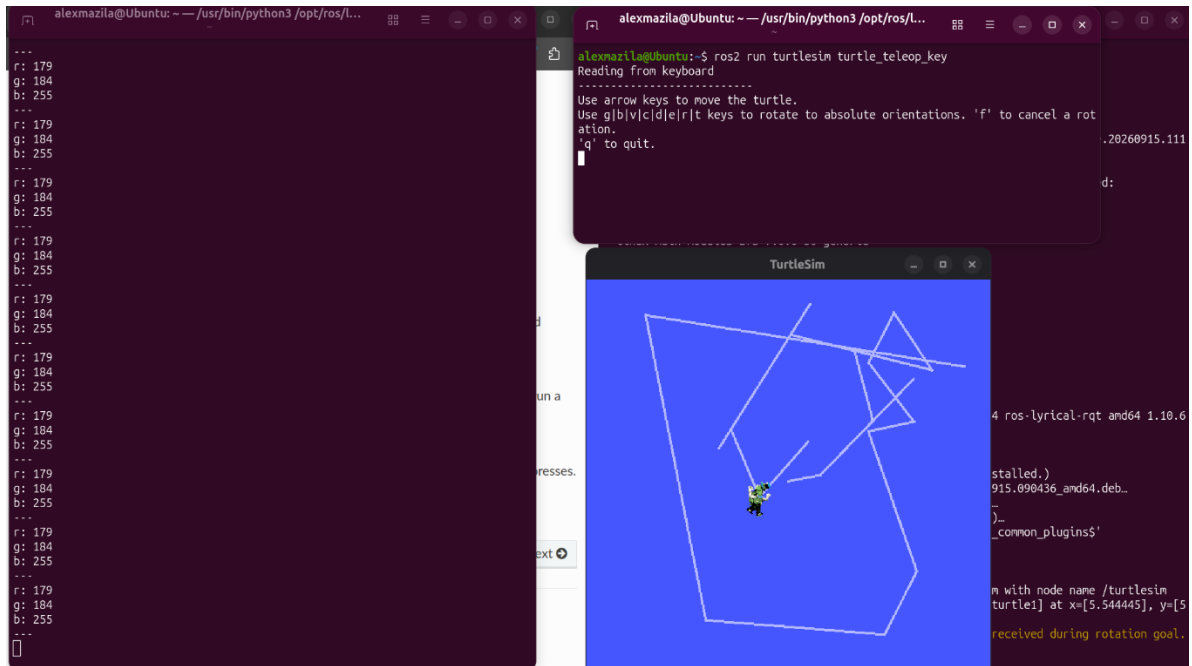
```
alexmazila@Ubuntu:~$ sudo apt update  
sudo apt install ros-lyrical-turtlesim  
[sudo: authenticate] Password:  
Hit:1 http://ua.archive.ubuntu.com/ubuntu resolute InRelease  
Hit:2 http://ua.archive.ubuntu.com/ubuntu resolute-updates InRelease  
Hit:3 http://ua.archive.ubuntu.com/ubuntu resolute-backports InRelease  
Hit:4 http://security.ubuntu.com/ubuntu resolute-security InRelease  
Get:5 https://packages.microsoft.com/repos/code stable InRelease [3,590 B]  
Get:6 https://packages.microsoft.com/repos/code stable/main amd64 Packages [30.5 kB]  
Hit:7 http://packages.ros.org/ros2/ubuntu resolute InRelease  
Fetched 34.1 kB in 1s (42.4 kB/s)  
All packages are up to date.  
ros-lyrical-turtlesim is already the newest version (1.10.9-1resolute.20260915.064816).  
ros-lyrical-turtlesim set to manually installed.  
The following packages were automatically installed and are no longer required:  
  linux-headers-7.0.0-30          linux-modules-7.0.0-30-generic  
  linux-headers-7.0.0-30-generic  linux-tools-7.0.0-30  
  linux-image-unsigned-7.0.0-30-generic  linux-tools-7.0.0-30-generic  
  linux-main-modules-zfs-7.0.0-30-generic  
Use 'sudo apt autoremove' to remove them.  
  
Summary:  
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 0
```

Запускаємо turtlesim і викликаємо teleop для керування черепахою стілками клавіатури:



2) Переглядаємо список node та topic. Вибираємо один із топіків для перегляду в реальному часі за допомогою `ros2 topic echo`





3) Створюємо власний пакет на збірці ament_python.

```
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_1/package_1$ ros2 pkg create --build
-type ament_python lab1
going to create a new package
package name: lab1
destination directory: /home/alexmazila/Documents/PDS_KSMm-26_Pavliuk/Lab_1/package_1
package format: 3
version: 0.0.0
description: TODO: Package description
maintainer: ['alexmazila <alexmazila@todo.todo>']
licenses: ['TODO: License declaration']
build type: ament_python
dependencies: []
creating folder ./lab1
creating ./lab1/package.xml
creating source folder
creating folder ./lab1/lab1
creating ./lab1/setup.py
creating ./lab1/setup.cfg
creating folder ./lab1/resource
creating ./lab1/resource/lab1
creating ./lab1/lab1/__init__.py
creating ./lab1/lab1/py.typed
creating folder ./lab1/test
creating ./lab1/test/test_copyright.py
creating ./lab1/test/test_flake8.py
creating ./lab1/test/test_mypy.py
creating ./lab1/test/test_pep257.py
creating ./lab1/test/test_xmllint.py

[WARNING]: Unknown license 'TODO: License declaration'. This has been set in the package.
.xml, but no LICENSE file has been created.
It is recommended to use one of the ament license identifiers:
Apache-2.0
BSL-1.0
BSD-2-Clause
BSD-3-Clause
GPL-3.0-only
LGPL-3.0-only
MIT
MIT-0
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_1/package_1$
```

Після збірки пакета створюємо build для подальшого використання функціоналу пакету.

```
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_1$ colcon build --packages-select py_pubsub
Starting >>> py_pubsub
Finished <<< py_pubsub [0.88s]

Summary: 1 package finished [0.98s]
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_1$
```

4) Створюємо окрему ноду Publisher publisher_member_function.py

Фрагмент коду, що надсилає повідомлення з періодичністю в 1 секунду:

```
def __init__(self):
    super().__init__('minimal_publisher')
    self.publisher_ = self.create_publisher(String, 'topic', 10)
    timer_period = 1 # seconds
    self.timer = self.create_timer(timer_period, self.timer_callback)
    self.i = 0

def timer_callback(self):
    msg = String()
    msg.data = 'Hello my friend!: %d' % self.i
    self.publisher_.publish(msg)
    self.get_logger().info('Publishing: "%s"' % msg.data)
    self.i += 1
```

Створюємо окрему ноду Subscriber subscriber_member_function.py

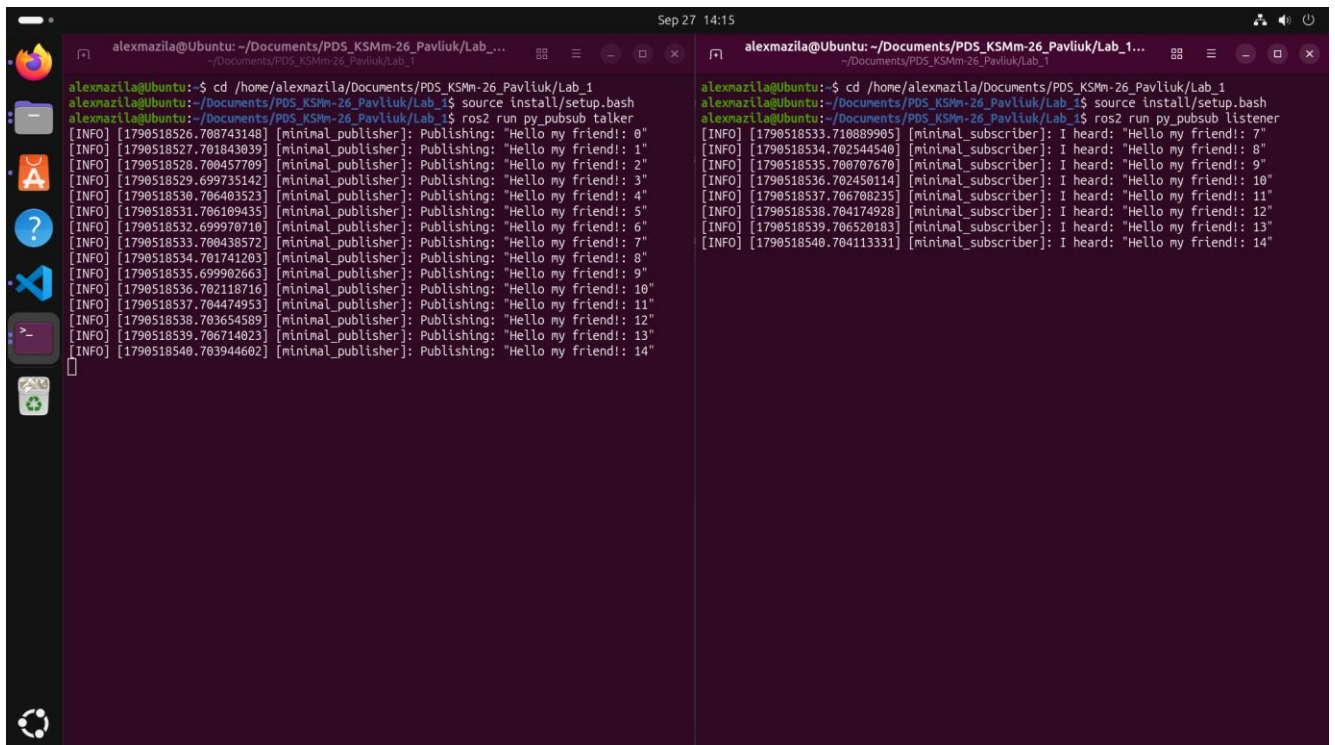
Фрагмент коду, що отримує повідомлення:

```
def __init__(self):
    super().__init__('minimal_subscriber')
    self.subscription = self.create_subscription(
        String,
        'topic',
        self.listener_callback,
        10)
    self.subscription # prevent unused variable warning
```

```
def listener_callback(self, msg):
```

```
    self.get_logger().info('I heard: "%s"' % msg.data)
```

5) Запускаємо ноди в окремих вікнах і фіксуємо успішну передачу та прослуховування.



The screenshot shows two terminal windows side-by-side. The left window shows a publisher node running 'talker' and the right window shows a subscriber node running 'listener'. Both windows display a series of messages being published and received, confirming successful communication.

```
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_1$ cd /home/alexmazila/Documents/PDS_KSMm-26_Pavliuk/Lab_1
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_1$ source install/setup.bash
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_1$ ros2 run py_pubsub talker
[INFO] [1790518526.708743148] [minimal_publisher]: Publishing: "Hello my friend!: 0"
[INFO] [1790518527.701843039] [minimal_publisher]: Publishing: "Hello my friend!: 1"
[INFO] [1790518528.700457709] [minimal_publisher]: Publishing: "Hello my friend!: 2"
[INFO] [1790518529.699735142] [minimal_publisher]: Publishing: "Hello my friend!: 3"
[INFO] [1790518530.706403523] [minimal_publisher]: Publishing: "Hello my friend!: 4"
[INFO] [1790518531.706109435] [minimal_publisher]: Publishing: "Hello my friend!: 5"
[INFO] [1790518532.699970710] [minimal_publisher]: Publishing: "Hello my friend!: 6"
[INFO] [1790518533.700438572] [minimal_publisher]: Publishing: "Hello my friend!: 7"
[INFO] [1790518534.701741203] [minimal_publisher]: Publishing: "Hello my friend!: 8"
[INFO] [1790518535.699902663] [minimal_publisher]: Publishing: "Hello my friend!: 9"
[INFO] [1790518536.702118716] [minimal_publisher]: Publishing: "Hello my friend!: 10"
[INFO] [1790518537.704474953] [minimal_publisher]: Publishing: "Hello my friend!: 11"
[INFO] [1790518538.703654509] [minimal_publisher]: Publishing: "Hello my friend!: 12"
[INFO] [1790518539.706714023] [minimal_publisher]: Publishing: "Hello my friend!: 13"
[INFO] [1790518540.703944602] [minimal_publisher]: Publishing: "Hello my friend!: 14"

alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_1$ cd /home/alexmazila/Documents/PDS_KSMm-26_Pavliuk/Lab_1
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_1$ source install/setup.bash
alexmazila@Ubuntu:~/Documents/PDS_KSMm-26_Pavliuk/Lab_1$ ros2 run py_pubsub listener
[INFO] [1790518533.710889905] [minimal_subscriber]: I heard: "Hello my friend!: 7"
[INFO] [1790518534.702544540] [minimal_subscriber]: I heard: "Hello my friend!: 8"
[INFO] [1790518535.700707670] [minimal_subscriber]: I heard: "Hello my friend!: 9"
[INFO] [1790518536.702450114] [minimal_subscriber]: I heard: "Hello my friend!: 10"
[INFO] [1790518537.706708235] [minimal_subscriber]: I heard: "Hello my friend!: 11"
[INFO] [1790518538.704174928] [minimal_subscriber]: I heard: "Hello my friend!: 12"
[INFO] [1790518539.706520183] [minimal_subscriber]: I heard: "Hello my friend!: 13"
[INFO] [1790518540.704113331] [minimal_subscriber]: I heard: "Hello my friend!: 14"
```

6) Висновки

На цій лабораторній роботі ми познайомились з середовищем ROS 2 для програмування роботизованих систем. За допомогою офіційного симулятора turtlesim зрозуміли поняття node та topic.

Створили власний пакет для розробки з двома простими нодами Publisher та Subscriber. Перший створює повідомлення з частотою раз на секунду, а другий підписаний на topic і прослуховує ці повідомлення.

Матеріали роботи було структуровано та опубліковано на GitHub https://github.com/alexmazila/PDS_KSMm-26_Pavliuk/tree/main